

Respecting Hardware Limits Inside the Safety Filter

How the FR3 actuator constraints are enforced *together* with the control-barrier condition, and why this preserves the safety guarantee

Sergi Goldsmits

June 2026

Abstract

This note documents how the planar FR3 collision-avoidance controller enforces the robot's physical actuator limits (joint-velocity saturation and joint-acceleration / slew limits) *without* breaking the forward-invariance guarantee of the control barrier function (CBF). The key design decision is that the actuator limits are not applied as a post-processing clamp on the certified command; they are imposed as box constraints inside the *same* quadratic program (QP) that enforces the CBF, so that the command which is published is exactly the command which is certified safe.

1 Two kinds of constraint

The controller must respect two qualitatively different requirements at every control tick.

Safety (geometric).

No link may enter the Capiasani forbidden cone of the obstacle. This is encoded by the barrier $h(q)$ ($h \geq 0$ is safe) and the CBF condition $\nabla h^\top \dot{q} \geq -\lambda h$. It is a property of the *world geometry*: h depends only on link and obstacle positions, not on the motor specifications.

Feasibility (hardware).

Each active joint has a maximum velocity $|\dot{q}_i| \leq \dot{q}_i^{\max}$ (saturation) and a maximum acceleration $|\ddot{q}_i| \leq \ddot{q}_i^{\max}$ (slew). Commanding outside these faults or damages the FR3. These are properties of the *actuator*, independent of where the obstacle is.

Both must hold for the velocity $\dot{q}$ that is actually sent to the robot.

2 Why a post-hoc clamp is unsafe

A natural but incorrect implementation enforces them in sequence: first solve the CBF projection to obtain a safe velocity $\dot{q}_{\text{cbf}}$, then clamp it to the actuator limits,

$$\dot{q}_{\text{pub}} = \text{clip}(\dot{q}_{\text{cbf}}, \dot{q}_{\text{prev}} - \ddot{q}^{\max} \Delta t, \dot{q}_{\text{prev}} + \ddot{q}^{\max} \Delta t).$$

The CBF guarantee certifies $\dot{q}_{\text{cbf}}$: it proves $\nabla h^\top \dot{q}_{\text{cbf}} \geq -\lambda h$. But the clamp changes the command *after* certification, so the robot receives $\dot{q}_{\text{pub}} \neq \dot{q}_{\text{cbf}}$. The realised rate of change of the barrier is $\nabla h^\top \dot{q}_{\text{pub}}$, which the clamp does not control and which can be $< -\lambda h$. The certificate applies to a command that was never sent. In practice this manifests as a first-order lag between the certified and applied velocities; while the slew-limited command catches up, h overshoots and dips below zero. The guarantee is only as good as the assumption that the certified command is the applied command, and a post-projection clamp violates exactly that assumption.

3 The fix: one QP, solved jointly

The limits are not applied to h and they are not applied after the filter. They are imposed as bound constraints in the *same* optimisation that enforces the CBF:

$$\boxed{\dot{q}_{\text{cmd}} = \arg \min_{\dot{q}} \frac{1}{2} \|\dot{q} - \dot{q}_{\text{nom}}\|^2 \quad \text{s.t.} \quad \underbrace{\nabla h^\top \dot{q} \geq -\lambda h}_{\text{CBF (safety)}}, \quad \underbrace{lb \leq \dot{q} \leq ub}_{\text{actuator box}}} \quad (1)$$

where the box is the intersection of the saturation and slew limits, evaluated against the previous command $\dot{q}_{\text{prev}}$ and the loop period Δt :

$$lb = \max(-\dot{q}^{\max}, \dot{q}_{\text{prev}} - \ddot{q}^{\max} \Delta t), \quad ub = \min(+\dot{q}^{\max}, \dot{q}_{\text{prev}} + \ddot{q}^{\max} \Delta t). \quad (2)$$

The saturation term gives a fixed box $[-\dot{q}^{\max}, \dot{q}^{\max}]$; the slew term gives a box of half-width $\ddot{q}^{\max} \Delta t$ *centred on the previous command*, because in one tick the velocity can move at most that far. Their intersection (2) is the set of physically admissible velocities this tick.

Geometric reading. The CBF constraint is a half-space (“stay on the safe side of a hyperplane with normal ∇h ”); the actuator box is a rectangle. The QP returns the point closest to the desired velocity $\dot{q}_{\text{nom}}$ that lies in the intersection of the half-space and the rectangle. There is a single feasible region and a single answer, and that answer is what gets published.

Proposition 1 (The published command is the certified command). *If (1) is feasible, its solution $\dot{q}_{\text{cmd}}$ satisfies both $\nabla h^\top \dot{q}_{\text{cmd}} \geq -\lambda h$ and $lb \leq \dot{q}_{\text{cmd}} \leq ub$ simultaneously. No further modification is applied before publishing, so the realised $\dot{h} = \nabla h^\top \dot{q}_{\text{cmd}} \geq -\lambda h$ and forward invariance of $\{h \geq 0\}$ holds for the actual robot command while respecting all actuator limits.*

4 Solving the QP

Problem (1) is a box-constrained least-squares projection with a single linear inequality, so it is solved exactly without an external solver. If the box projection of the nominal already satisfies the CBF, it is optimal and returned directly (the filter is inactive and minimally invasive). Otherwise the CBF constraint is active, $\nabla h^\top \dot{q} = -\lambda h$, and the KKT conditions give the separable form

$$\dot{q}_i(\mu) = \text{clip}(\dot{q}_{\text{nom},i} + \mu [\nabla h]_i, lb_i, ub_i), \quad \mu \geq 0,$$

where the scalar multiplier μ is found by a one-dimensional root solve of $\nabla h^\top \dot{q}(\mu) = -\lambda h$. The map $\mu \mapsto \nabla h^\top \dot{q}(\mu)$ is monotone non-decreasing, so a bisection converges reliably; the cost is a handful of clip-and-dot operations per tick, negligible at 100 Hz.

5 Infeasibility is reported, not hidden

The slew box has half-width $\ddot{q}^{\max} \Delta t$. If the manipulator is approaching the obstacle fast enough that the required braking velocity lies outside this box, no admissible $\dot{q}$ can satisfy the CBF in that tick: (1) is infeasible. This is not a software fault—it is the physical statement that the actuator cannot decelerate quickly enough. The implementation detects this case, returns the admissible velocity that comes closest to satisfying the CBF (the safest reachable command, i.e. the box point that maximises $\nabla h^\top \dot{q}$), and raises a `feasible=False` flag in the logged/published status. The principled remedy is to enlarge the safety margin so braking begins earlier: reduce the class- $\mathcal{K}$ gain λ (a smaller λ is *more* conservative—it forces earlier, gentler braking) or increase the obstacle inflation r_{eff} .

Remark 1 (What is *not* done after the QP). No filtering, smoothing, or clamping is applied to $\dot{q}_{\text{cmd}}$ after (1); any such step would again sit outside the certificate. If output smoothing is desired it must be applied to the *nominal* $\dot{q}_{\text{nom}}$ *before* the QP, so the QP certifies the smoothed command.

Summary

- The barrier h stays purely geometric; hardware limits never enter h .
- Hardware limits enter as a *box* on the velocity in the *same* QP as the CBF constraint (1)–(2): saturation as a fixed box, acceleration as a slew box centred on the previous command.
- Because safety and feasibility are solved jointly, the published command equals the certified command; forward invariance holds for the command actually applied.
- When the actuator physically cannot brake fast enough, the QP is infeasible and this is flagged; the cure is more margin (smaller λ , larger r_{eff}), not post-processing.